Royalty Hub — Technical Report (Final Version of the Product)

MUNEZERO Hubert · BSc Software Engineering, African Leadership University · Supervisor: Emmanuel Annor · 2026-10-25
Repository: https://github.com/Hubert2c/Royalties-claim · Walkthrough video: see the link in the README

Product. Royalty Hub is a government-administered (RIPO) platform that records every public use of music in Rwanda — radio, TV, venues, concerts, adverts, digital — collects licence fees, and pays each songwriter, composer, publisher and performer their confirmed share, with a public dashboard and a tamper-evident audit log. It has five portals (government console, auditor view, rights holders, licensees, public site) and an open API for monitors and partner systems. Stack: Node.js 22, Express 5, SQLite (prototype) / PostgreSQL (production), plain HTML/CSS/JavaScript front-end, node:test, ESLint, autocannon, Playwright.

Walkthrough video map. The narrated video shows every core feature working, the backend and data storage, wrong inputs, the testing strategies, and performance and compatibility results:

Time	Video chapter	Time	Video chapter
0:00	Capstone demo · African Leadership University	4:46	10 · The monthly distribution run
0:17	The problem	5:14	11 · The artist sees exactly where the money came from
0:41	1 · Public transparency dashboard	5:30	12 · Nobody can secretly change the records
1:19	2 · A radio station applies for a licence	5:49	13 · Behind the screens: the backend
1:44	3 · The government approves and invoices	9:05	14 · Testing with different inputs
2:04	4 · The station pays; the code becomes valid	9:52	15 · Testing strategies
2:40	5 · How the government knows a song was played	11:12	16 · Performance and compatibility
3:17	6 · The station also reports what it played	12:29	17 · Extras
3:35	7 · An artist registers a song and its splits	12:41	What is real, what is simulated
4:13	8 · The co-writer confirms her share	13:07	Royalty Hub
4:27	9 · Claiming an unclaimed play		

1. Scope alignment and feature validation

Planned feature (proposal §3.3)	Intended purpose	Status	Evidence
FR1 Register rights holders, works, recordings, ISWC/ISRC/IPI	One national registry of who owns what	**Completed**	routes/rightsholder.js; video 3:35
FR1 Verify identity (NIDA)	Stop false ownership claims	**Partial**	email login + RIPO review of claims; NIDA link planned for pilot
FR2 Declare and confirm splits; reject ≠ 100%	Co-writers paid fairly; no lost shares	**Completed**	RoyaltyEngine.splitsAreComplete; tests: splits ≠ 100%
FR3 Licence applications and tariff invoices	Every music user pays by approved rules	**Completed**	LicensingService, TariffRule subclasses
FR4 Ingest fingerprints, play logs, set lists, cue sheets, CSV/DDEX	Know every use of music	**Completed**	UsageService, CsvUsageParser, JsonUsageParser
FR5 Identify broadcast songs by fingerprint	Detect radio/TV plays without trusting logs	**Completed**	FingerprintMatcher; tools/monitor-report.js (fpcalc)
FR6 Calculate royalties after a published fee (≤ 30%)	Exact, transparent shares	**Completed**	RoyaltyEngine, MoneyAllocator; 500 property tests
FR7 Government approves tariffs, licences, runs	Public control before money moves	**Completed**	routes/admin/*; role tests (auditor read-only)
FR8 Itemised statements and payouts (MoMo / bank)	Artists see and receive their money	**Partial**	statements complete; payouts via PaymentGateway in simulated mode until provider agreements
FR9 Public unclaimed list and claims	Unknown plays are not lost	**Completed**	/api/public/unclaimed; claims workflow; video 4:27
FR10 Disputes	Fair resolution of disagreements	**Completed**	routes/disputes.js, admin claims & disputes
FR11 Audit log of every action	Tamper-evident accountability	**Completed**	AuditLog (SHA-256 chain); tamper test
FR12 Public dashboard	Trust: anyone sees the money flow	**Completed**	/public; video 0:41
FR13 Open APIs (ONDC approach)	Stations, monitors, partners plug in	**Completed**	/api/open/v1, API keys, /docs
NFR Security & privacy	Protect people and payment data	**Completed**	JWT, 4 roles, bcrypt, AES-256-GCM, CORS, Helmet, rate limit
NFR Accuracy (exact sums; ≥ 90% fingerprint precision)	Correct payments	**Completed**	exact in all tests; precision 100.0% on synthetic clips
NFR Performance (run < 1 min; API < 2 s)	Usable at national scale	**Completed**	100,000-play run in 51.8 ms; p99 < 200 ms at 50 users
NFR Usability (SUS ≥ 68)	Artists without technical training	**Partial**	responsive UI, plain language; SUS survey with artists planned for pilot

NFR Availability 99%	Financial service must be up	**Partial**	Docker/Vercel/Postgres ready; not yet hosted long enough to measure
Live MoMo, Irembo, RURA, RRA, CISAC links	Collect and enforce nationally	**Not implemented**	out of prototype scope: needs agreements (Phase 2–4); adapters ready

Scope changes and justification. All 13 functional requirements are implemented. Three items are deliberately *partial* because they depend on institutions, not code: live payouts and identity checks need agreements with MTN/Airtel, banks and NIDA; usability (SUS) and availability must be measured with real users and a long-running deployment. They were designed as extension points (e.g. `PaymentGateway.livePayout()` refuses with a clear `NotConfiguredError` instead of pretending), so the pilot (Phase 2) only adds provider adapters. International links (CISAC, EAC) belong to Phase 4 of the rollout. Four small features were *added* beyond the proposal (notifications, a live airplay feed, an earnings estimator and printable certificates) to make the portals usable for non-technical artists.

2. Technical implementation

Architecture. Layered, as in proposal Figure 3: **presentation** (five static portals) → **API** (Express; security middleware: JWT, four roles, same-origin-aware CORS, Helmet headers, rate limiting) → **thin routes** (`routes/`, the admin API split into seven files) → **services** (`LicensingService`, `UsageService`, `DistributionService`, `NotificationService`, `AuditLog`) built once in `lib/services.js` (composition root) → **domain classes** (`RoyaltyEngine`, `FingerprintMatcher`, `MoneyAllocator`, tariff rules, gateways) → **data** (`DatabaseAdapter`, 18 tables from the ERD, 20 indexes).

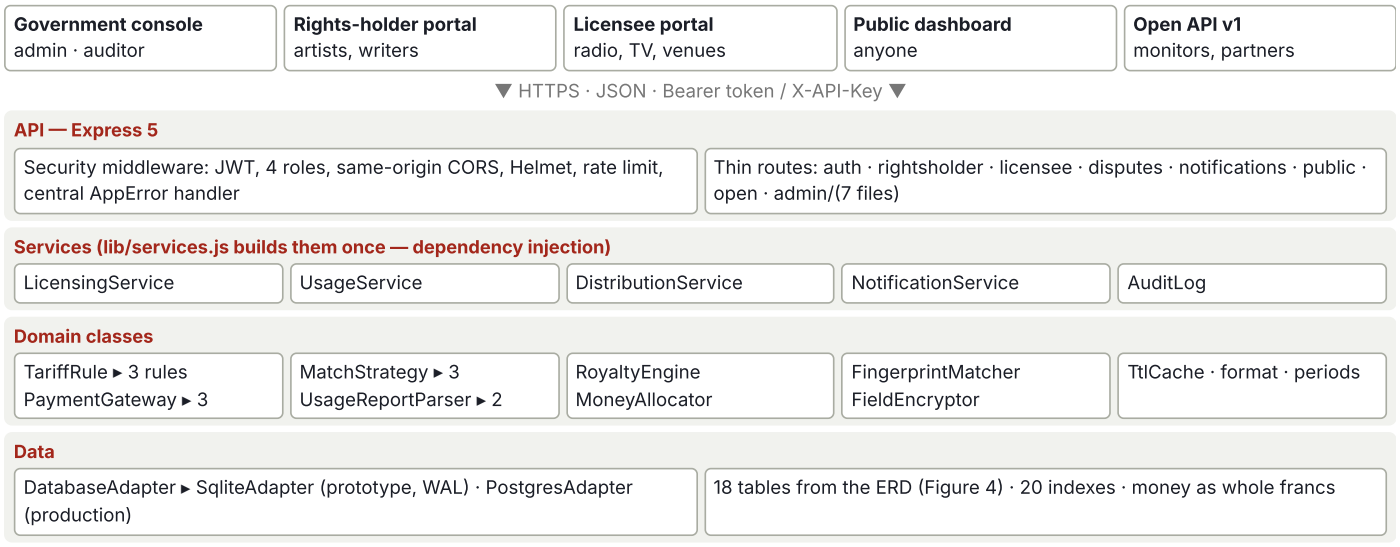


Figure 1. Layered architecture of the final version (▸ = subclasses).

Frontend–backend interaction. The portals call a JSON REST API with a bearer token; machines use `X-API-Key` (stored only as a SHA-256 hash). Example: a station's play log (CSV) → `CsvUsageParser` → `UsageService` matches each row (fingerprint → ISRC → title + artist) → saved as matched or unmatched → the monthly `DistributionService` run → statements and public dashboard. **Data handling:** money in whole francs (integers), payout numbers encrypted with AES-256-GCM (only the last 4 digits shown), passwords bcrypt-hashed, every royalty line linked to the exact play, plays with no owner kept on a public list.

Algorithms and problem solving (measured on a 2-core server)

Algorithm	How it works	Complexity	Measured
Fingerprint matching	Slide the clip over each registered fingerprint; XOR 32-bit values and count differing bits with a branch-free SWAR popcount; match if ≥ 65 % of bits agree. Parsed fingerprints cached as `Uint32Array`.	O(R·L·S)	10 songs 0.24 ms · 1,000 songs 21.44 ms · 5,000 songs 125.55 ms (linear). Precision **100.0 %**, recall 100.0 % on 600 clips vs 1,000 songs.
Largest-remainder split	Whole-franc part of each exact share, then leftover francs to the largest fractions (ties by position → deterministic). Used twice: plays → equal value, play → co-writer shares.	O(n log n)	1,000,000 shares in 332.56 ms; always exact (500 random property tests). Month of 100,000 plays: 51.8 ms (target: < 1 min).
SHA-256 hash chain	Each audit entry hashes the previous hash + its own data; verification recomputes the chain and names the first broken entry. Writes are queued so the order is strict.	O(n)	Verify 10,000 entries in 20.8 ms; edited and deleted entries detected (tests).

3. Code quality and OOP

Abstract base	Subclasses	Design and principle
`PaymentGateway`	MobileMoneyGateway, BankTransferGateway, IremboGateway	Template Method + Factory: same collect()/payout() steps; each channel validates payers its own way.
`MatchStrategy`	FingerprintMatchStrategy, IsrcMatchStrategy, TitleArtistMatchStrategy	Strategy: UsageMatcher tries them in order; a new method = one new class (open/closed).

`TariffRule`	FlatAnnualRule, PercentGrossRule, PerUnitRule	Factory picks the rule from the tariff basis; no switch statements in callers.
`UsageReportParser`	CsvUsageParser, JsonUsageParser	Every format returns the same row shape (Liskov substitution).
`DatabaseAdapter`	SqliteAdapter, PostgresAdapter	Adapter: services never know which database runs (dependency inversion).
`AppError`	ValidationError, AuthenticationError, ForbiddenError, NotFoundError, ConflictError, NotConfiguredError	Each error carries its HTTP status; one error handler for the whole API.

- **Encapsulation:** each class owns its data (e.g. `FieldEncryptor` keeps the key private; `AuditLog` owns the chain and its write queue). **Abstraction:** abstract bases throw if instantiated (`new.target` check, tested).
- **SOLID:** single responsibility (routes only translate HTTP; services hold business rules; the engine is pure maths), open/closed (strategies, rules, gateways), dependency inversion and **dependency injection** (services receive `db`, engine, gateways in the constructor; tests inject a fake gateway).
- **Organisation:** `lib/` by domain (royalties, usage, fingerprint, licensing, payments, tariffs, audit, security, notifications, util), `db/` (adapters, schema, seed), `routes/`, `tests/` (unit, integration, performance, compatibility). The former 512-line admin router and 558-line database file were split; no server file is over ~400 lines.
- **Readability:** classes in PascalCase, methods in camelCase, every module opens with a header explaining its purpose and the requirement it covers; JSDoc on public methods; ESLint (recommended rules + `eqeqeq`, `prefer-const`) reports 0 problems.

4. Testing approaches and results

Strategy	Why selected	How conducted	Results
Unit (29 tests)	Money must be exact; algorithms first.	`node:test`; classes alone; fakes injected; 500 random property tests; edge cases (0 francs, reversed dates, BOM in CSV).	29/29 pass.
Integration / API (18 tests)	Front-end and back-end meet in the API; roles and wrong inputs.	Real server + throw-away database: full flow licence → payment → usage → run → payout → statement; 40+ wrong inputs (bad logins, forged tokens, splits ≠ 100 %, bad MoMo number, paying twice, empty CSV, bad period, foreign origins).	18/18 pass; whole suite 47/47 in ≈ 3 s.
Load and stress	National scale: many stations at once.	autocannon, 10/50/100 users × 4 endpoints, CPU/RAM sampled; 100,000-play database; flood against the rate limit.	See §5; flood: 199 served, 801 refused with 429.
Compatibility and performance	Artists on phones; slow offices.	Playwright: 5 screens × 4 portals; DevTools CPU (×4, ×6) and network (Fast/Slow 3G) throttling.	20/20 combinations work, 0 px sideways scroll.

Improvements made after testing (before → after, 50 users): usage reports 39 → 554 req/s (SQLite WAL + NORMAL sync); fingerprint matching 183 → 1268 req/s (parsed-fingerprint cache); dashboards that slowed as the plays table grew got 20 indexes and a 3-second cache; concurrent audit writes could clash → write queue; and the compatibility test found that **login failed when the site was opened by IP address** (e.g. a phone on the same Wi-Fi) → same-origin CORS rule plus a regression test.

5. Performance and compatibility

Server: Intel(R) Xeon(R) Processor, 2 cores, 8 GB RAM, Linux 6.18.44-fc-v153, Node v22.23.4; 10 s per scenario. Cells: requests/s and p99 latency (ms), before → **after** improvements.

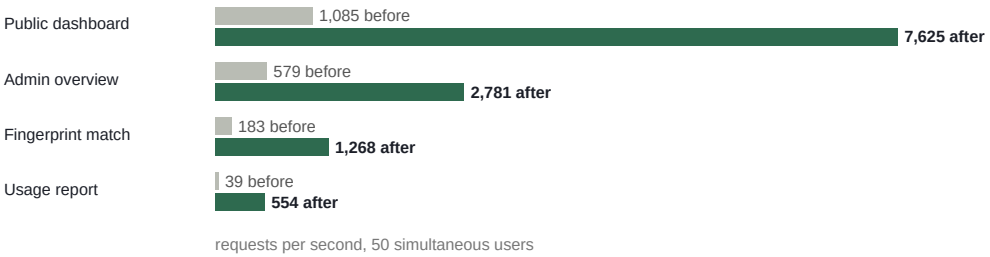


Figure 2. Throughput before and after the improvements found by testing.

Endpoint	10 users		50 users		100 users	
	req/s	p99 ms	req/s	p99 ms	req/s	p99 ms
Public dashboard	1,316 → **7,630**	16 → **3**	1,085 → **7,625**	69 → **10**	744 → **7,832**	2,270 → **18**
Admin overview (12 queries)	1,172 → **2,835**	19 → **6**	579 → **2,781**	152 → **28**	355 → **2,898**	3,142 → **45**
Fingerprint match	189 → **1,309**	104 → **18**	183 → **1,268**	3,593 → **62**	180 → **1,311**	4,519 → **105**
Usage report, 10 plays	37 → **571**	868 → **33**	39 → **554**	6,103 → **172**	38 → **546**	6,137 → **1,914**

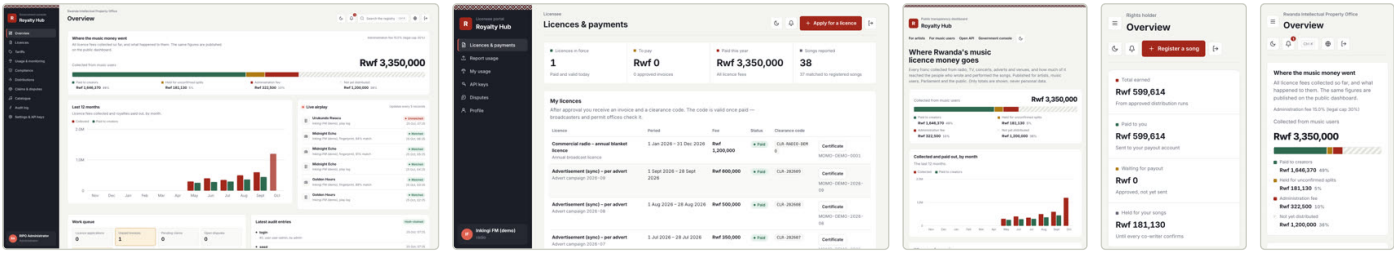
Big database: 100,000 extra plays inserted through the open API in 12.1 s; afterwards the admin overview still served 2,839 req/s (p99 27 ms) at 50 users. Errors: under 0.1 % at 100 users (requests cut when each timed run stops); a repeat run had 0 errors in 4,506 requests.

Hardware and network profiles (Chromium, 1366×768)

Profile	Page	First paint	Loaded	Usable*	JS MB	KB
High-end (no throttling)	Public dashboard	116	80	**608**	2.3	78
High-end (no throttling)	Government console	80	72	**756**	2.9	127
Mid-range (CPU ×4, Fast 3G)	Public dashboard	776	939	**1,598**	2.3	78
Mid-range (CPU ×4, Fast 3G)	Government console	904	1,055	**2,296**	2.9	127
Low-end (CPU ×6, Slow 3G)	Public dashboard	2,104	2,576	**3,481**	2.3	78
Low-end (CPU ×6, Slow 3G)	Government console	2,472	2,911	**5,093**	2.4	127

Times in ms. *Usable = page shown, logged in, data loaded. The front-end has no framework: ≈ 80–130 KB per page, < 3 MB memory.

Screens and browsers



Desktop Laptop Tablet iPhone 13 Pixel 5

All four portals work on desktop 1920×1080, laptop 1366×768, iPad Mini, iPhone 13 and Pixel 5 in chromium 153.0.8010.12 — the engine of Chrome, Edge, Opera and Samsung Internet — with no JavaScript errors and no sideways scrolling. Firefox and Safari engines were not installed on the test machine; `BROWSERS=chromium,firefox,webkit npm run test:browsers` runs the same checks there. Software: Node 22.5+ (SQLite via better-sqlite3 or built-in node:sqlite), Windows/macOS/Linux, Docker.

6. Limitations and next steps

Fingerprint accuracy was measured on synthetic fingerprints with random noise; real broadcast recordings (with `fpcalc`) must be measured in the pilot. Payments run in simulated mode; tariff amounts are demo values until the 2026 Ministerial Order figures are entered. Next: connect MoMo/Airtel/Irembo adapters, deploy on government-controlled PostgreSQL, run a SUS usability study with artists, and test Firefox/Safari on physical devices.